

Anomaly Detection — Complete Notes & Concepts

This document is a compact but comprehensive set of notes on anomaly detection for machine learning, data science, and real-world analytics. It covers core concepts, mathematical intuition, common algorithms, evaluation methods, practical workflows, challenges, and implementation guidance.

Goal: identify observations, events, patterns, or behaviors that are rare, unexpected, inconsistent with the majority of the data, or potentially harmful/interesting.

1. What is Anomaly Detection?

Anomaly detection is the task of finding data points or patterns that deviate significantly from normal behavior.

Anomalies are also called **outliers**, **novelties**, **exceptions**, or **rare events** depending on context.

Unlike standard classification, anomaly detection often has very few labeled anomaly examples, so methods are commonly unsupervised or semi-supervised.

Examples:

- Credit card fraud transactions
- Intrusion attempts in computer networks
- Faults in machines and industrial sensors
- Unusual patient health measurements
- Suspicious insurance claims
- Defective products in manufacturing
- Abnormal user behavior in web apps or social media

2. Why anomaly detection matters

Many important business and safety problems are not about predicting the common case, but about detecting the unusual case quickly.

Anomalies are often high-impact: fraud, outages, cyberattacks, failures, medical emergencies, quality issues, and financial losses.

3. Outlier vs Novelty Detection

Outlier detection: detect unusual points in a dataset that may already contain anomalies mixed with normal data.

Novelty detection: learn normal behavior from clean normal training data, then flag future observations that do not fit that learned normal pattern.

In practice, the terms are sometimes used interchangeably, but the training setup is different.

4. Types of anomalies

Point anomalies: a single observation is unusual compared with the rest of the data.

Contextual anomalies: an observation is anomalous only in a specific context such as time, location, season, or user segment.

Collective anomalies: a group or sequence of points is anomalous even if individual points are not extreme.

Local anomalies: unusual relative to nearby points or local neighborhood.

Global anomalies: unusual relative to the full dataset distribution.

5. Supervision settings

Supervised anomaly detection: both normal and anomaly labels are available. This is rare but possible in mature fraud or quality pipelines.

Unsupervised anomaly detection: no labels. The method assumes anomalies are rare and different from the majority.

Semi-supervised / one-class setting: train mostly on normal data and learn the boundary of normal behavior. New points outside that region are flagged.

6. Core intuition

Most anomaly detection methods try to assign an **anomaly score** to each observation. Higher score means more abnormal.

Scores can be based on:

- distance from other points
- low estimated probability density
- isolation depth in random trees
- reconstruction error from an autoencoder/PCA
- deviation from forecast or expected pattern
- graph irregularity or rule violation

7. General pipeline

- 1) Understand the domain and define what counts as an anomaly.
- 2) Gather data and create features.
- 3) Clean missing values, duplicates, and obvious data issues.
- 4) Scale or transform variables when needed.
- 5) Choose algorithm based on data type and labels.
- 6) Train model and compute anomaly scores.
- 7) Set threshold for alerts.
- 8) Evaluate with appropriate metrics and human review.
- 9) Deploy monitoring and retraining loop.

8. Challenges in anomaly detection

Anomalies are rare, so class imbalance is severe.

Labels are expensive and often incomplete.

Normal behavior changes over time (concept drift).

Noise and bad data can look like anomalies.

False positives can overwhelm analysts.

The meaning of anomaly depends heavily on business context.

9. Data preprocessing for anomaly detection

Handle missing values carefully. Missingness itself can be informative.

Remove duplicates if they are accidental; keep them if repetition is meaningful.

Scale features for distance-based methods using standardization or robust scaling.

Encode categorical variables with care (one-hot, target encoding, embeddings depending on setting).

Create time features for temporal data: hour, weekday, season, lag, rolling mean, rolling std.

Reduce skew with log transform or Box-Cox/Yeo-Johnson when appropriate.

Use robust statistics when heavy tails exist.

10. Feature engineering ideas

Ratios, differences, rates of change, rolling deviations, z-scores per entity, frequency features, graph degrees, device/user aggregates, and interaction terms can improve anomaly detection.

For transactional data, engineer customer history, merchant frequency, average spend, time since last transaction, location deviation, etc.

11. Statistical methods overview

Statistical approaches assume the data follows a probability distribution or a stable baseline. Points in low-probability regions are anomalous.

12. Z-score method

For a numeric feature with mean μ and standard deviation σ , the z-score is:

$$z = (x - \mu) / \sigma$$

If $|z|$ is large (for example > 3), the point may be anomalous.

This works best when the data is roughly Gaussian and univariate.

13. Modified Z-score

Standard z-score is sensitive to outliers because mean and standard deviation are not robust.

Modified z-score uses the median and MAD (Median Absolute Deviation):

$$\text{MAD} = \text{median}(|x_i - \text{median}(x)|)$$

$$\text{Modified } z = 0.6745 * (x - \text{median}) / \text{MAD}$$

This is more robust for skewed or contaminated data.

14. IQR method

Interquartile Range: $\text{IQR} = Q3 - Q1$.

Common rule: flag $x < Q1 - 1.5 * \text{IQR}$ or $x > Q3 + 1.5 * \text{IQR}$.

Useful for simple univariate screening and boxplots.

15. Gaussian / parametric density models

Assume data comes from a Gaussian distribution (univariate or multivariate).

Estimate parameters (mean vector and covariance matrix), then compute how likely each point is under the model.

Points with very low probability density are anomalies.

16. Mahalanobis distance

For multivariate Gaussian-style data, distance from the mean should consider covariance and correlation among features.

Mahalanobis distance:

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

Large Mahalanobis distance suggests anomaly.

It works best when covariance is estimated reliably and the distribution is not extremely non-Gaussian.

17. KDE (Kernel Density Estimation)

KDE estimates the probability density of data without assuming a strict parametric form.

Low-density regions correspond to anomalies.

Bandwidth selection is critical. KDE can struggle in high dimensions because of the curse of dimensionality.

18. Distance-based methods

The intuition: normal points tend to have neighbors nearby, while anomalies are far from others.

Distance can be Euclidean, Manhattan, cosine, or domain-specific.

19. k-Nearest Neighbors (kNN) anomaly detection

Compute the distance from each point to its k nearest neighbors.

Common anomaly scores:

- distance to the k-th nearest neighbor
- average distance to the k nearest neighbors

Large distance implies anomaly.

kNN is simple and intuitive but can be expensive on large/high-dimensional data.

20. Local Outlier Factor (LOF)

LOF compares the local density of a point with the local densities of its neighbors.

If a point lies in a region much less dense than its neighbors, it gets a high LOF score.

LOF is good for local anomalies that may not be global extremes.

21. Clustering-based methods

Fit clusters to normal structure. Points that do not belong well to any cluster, or belong to tiny sparse clusters, are flagged.

Distance to cluster centroid or cluster assignment confidence can be used as an anomaly score.

22. K-Means for anomaly detection

Train K-Means on the data.

Use distance to assigned centroid as anomaly score.

Far-away points are suspicious.

Works best when normal data forms compact spherical-ish groups.

23. DBSCAN for anomaly detection

DBSCAN groups dense regions and labels sparse points as noise.

Noise points are natural anomaly candidates.

Useful when clusters have arbitrary shapes, but parameter tuning (eps, min_samples) matters.

24. Isolation Forest

Isolation Forest is one of the most widely used unsupervised anomaly detection algorithms.

It isolates points by recursively splitting data with random features and random split values.

Anomalies tend to be isolated in fewer splits because they are rare and different.

The average path length across trees becomes the anomaly signal: shorter path = more anomalous.

25. Why Isolation Forest works well

No need to estimate full density.

Works reasonably well on high-dimensional data.

Scales better than many distance-based methods.

Handles mixed feature patterns better than simple univariate rules.

26. One-Class SVM

One-Class SVM tries to learn a boundary around normal data in feature space.

Points outside that learned boundary are labeled anomalous.

Kernel choice matters; RBF is common.

It can work well on moderate-size data but may become expensive on large datasets.

27. Elliptic Envelope

Fits a robust covariance estimate assuming data is Gaussian-like.

Then uses Mahalanobis distance to detect outliers.

Best for approximately elliptical distributions.

28. PCA for anomaly detection

Principal Component Analysis compresses data into a lower-dimensional subspace capturing the main variance of normal data.

Anomalies often reconstruct poorly from the principal components.

Two common scores:

- reconstruction error
- distance in residual subspace

29. Autoencoders

Autoencoders are neural networks trained to reconstruct their input.

If trained mostly on normal data, they learn to reconstruct normal patterns well and anomalous patterns poorly.

Reconstruction error can be used as anomaly score.

Variants include denoising autoencoders, variational autoencoders, LSTM autoencoders for sequences, and convolutional autoencoders for images.

30. Time-series anomaly detection

Time-series anomalies are extremely common in monitoring, finance, IoT, and operations.

An anomaly may be a spike, drop, level shift, variance change, missing pattern, trend break, or unusual sequence.

31. Time-series approaches

Forecasting-based: predict expected future value and flag large residuals.

Statistical decomposition: remove trend/seasonality, analyze residuals.

Change point detection: find structural shifts.

Sequence models: LSTM/Transformer/Temporal CNN/autoencoder.

Control-chart style methods: Shewhart, EWMA, CUSUM.

32. Forecast residual method

Fit a forecasting model such as ARIMA, Prophet, Exponential Smoothing, or an LSTM.

Compute residual = actual – predicted.

Large absolute residuals or residuals outside confidence intervals indicate anomalies.

33. STL decomposition idea

Decompose series into Trend + Seasonality + Residual.

Analyze residual component for spikes or abnormal patterns.

This helps avoid flagging normal seasonal peaks as anomalies.

34. Change point detection

Change point methods detect when the data generating process changes, for example a new mean, variance, slope, or distribution.

Examples: CUSUM, Bayesian change point detection, ruptures library methods.

35. Deep learning for anomaly detection

Useful when data is complex: images, audio, long sequences, logs, multivariate sensors, graphs.

Common approaches:

- autoencoders / VAEs
- predictive models with residual scoring
- contrastive/self-supervised representations
- GAN-based anomaly detection
- graph neural network based anomaly detection

36. Graph anomaly detection

Used when data naturally forms nodes and edges: users-transactions-devices, social networks, communication networks, supply chains.

Anomalies may be unusual nodes, edges, communities, or subgraphs.

Features include degree, edge weight, neighborhood similarity, temporal evolution, and learned graph embeddings.

37. NLP / text anomaly detection

Detect unusual documents, log messages, customer support tickets, or spam/phishing content.

Represent text using TF-IDF, embeddings, sentence transformers, or language-model features.

Then apply clustering, density, one-class, or sequence anomaly methods.

38. Computer vision anomaly detection

In manufacturing and medical imaging, normal images are common while defects are rare.

Approaches include autoencoders, feature reconstruction, patch-based methods, contrastive pretraining, and vision-transformer embeddings.

39. Root cause vs anomaly detection

Anomaly detection answers: *What looks unusual?*

Root cause analysis answers: *Why did it happen?*

A production system often needs both: detect the anomaly and then explain likely drivers.

40. Interpreting anomaly scores

A raw score is not a final business decision.

Convert scores into categories such as normal / warning / critical based on thresholds, risk appetite, analyst capacity, and cost of mistakes.

41. Threshold selection

Thresholding is often the hardest part.

Common strategies:

- fixed percentile (top 1% highest scores)

- score calibrated on validation data
- threshold from expected contamination rate
- threshold tuned to maximize F1 or business utility on labeled validation set
- dynamic thresholds for time-series or per-entity baselines

42. Contamination parameter

Several libraries ask for **contamination**, the expected fraction of anomalies in the data.

It helps convert raw scores into labels.

If contamination is wrong, alert rate may be too high or too low.

43. Evaluation challenge

Accuracy is usually a bad metric because anomalies are rare.

A model can get very high accuracy by predicting everything as normal.

44. Better evaluation metrics

Precision: of flagged anomalies, how many were truly anomalous?

Recall: of true anomalies, how many did we catch?

F1-score: harmonic mean of precision and recall.

PR-AUC is often more informative than ROC-AUC under heavy class imbalance.

Average precision, top-k precision, alert rate, time-to-detect, and analyst feedback yield are also important in production.

45. Confusion matrix in anomaly detection

TP: anomaly correctly flagged

FP: normal point incorrectly flagged

TN: normal correctly ignored

FN: anomaly missed

Business cost often makes FP and FN very asymmetric.

46. Cost-sensitive thinking

In fraud detection, a false negative may be costly because fraud is missed.

In monitoring systems, too many false positives cause alert fatigue.

Choose threshold and model with the operational cost in mind, not only a benchmark score.

47. Cross-validation considerations

When labels exist, use stratified validation if possible.

For time-series, never randomly shuffle across time if future information leaks into training.

Use rolling or time-based splits for temporal anomaly detection.

48. Explainability techniques

Explainability is crucial because users need to trust alerts.

Possible tools:

- feature contribution analysis
- SHAP for supervised models or score surrogates
- nearest normal examples vs anomalous examples
- reconstruction-error breakdown per feature
- rule extraction / threshold explanation

49. Common pitfalls

Treating data errors as business anomalies without validation.

Using future information in features for time-series.

Assuming all outliers should be removed before modeling.

Using Euclidean distance on unscaled features.

Expecting one algorithm to work for every anomaly type.

Choosing threshold without considering alert volume.

50. Outlier removal vs anomaly detection

Outlier removal is a preprocessing choice: removing unusual points before fitting another model.

Anomaly detection is itself the main objective: finding unusual events because they matter.

Not every outlier should be removed; some are the most important cases.

51. Robust statistics and robust models

Median, MAD, robust covariance, Huber loss, and quantile-based methods are useful when data contains contamination.

Robust methods reduce the influence of extreme points on the model.

52. High-dimensional issues

In high dimensions, distance concentration makes many distance-based methods weaker.

Feature selection, dimensionality reduction, representation learning, and domain-specific embeddings become important.

53. Imbalanced labeled anomaly detection

If you have anomaly labels, you can train supervised models such as gradient boosting, random forests, XGBoost, CatBoost, or neural networks.

Use imbalance handling such as class weights, focal loss, anomaly oversampling with caution, and threshold tuning.

54. Ensemble anomaly detection

Combining multiple detectors often works better than relying on one.

Examples: average normalized scores from Isolation Forest + LOF + Autoencoder, or use rank aggregation.

Ensembles improve robustness across anomaly types.

55. Active learning and human-in-the-loop

Analysts can review top alerts and provide feedback.

That feedback can be used to tune thresholds, retrain models, or build supervised anomaly classifiers.

Human review is often the bridge from unsupervised scoring to business value.

56. Streaming anomaly detection

In streaming systems, data arrives continuously and models must update or score online.

Needs low latency, memory efficiency, and drift handling.

Common ideas: rolling baselines, incremental statistics, online forests, online change detection, window-based scoring.

57. Concept drift and data drift

Data drift: input distribution changes.

Concept drift: the definition of normal/anomalous changes or the relationship between features and labels changes.

Monitor both and retrain when needed.

58. Domain examples

Fraud: unusual spend, merchant, location, device, sequence.

Cybersecurity: impossible login times, privilege escalation, lateral movement, suspicious traffic.

Manufacturing: vibration anomalies, temperature spikes, visual defects.

Healthcare: abnormal vitals, rare lab combinations, unusual disease progression.

Retail: sudden demand spikes, inventory discrepancies.

Cloud/DevOps: CPU spikes, latency shifts, error-rate bursts.

59. Choosing an algorithm — quick guide

If you only have tabular unlabeled data: start with Isolation Forest, LOF, and simple statistical baselines.

If you have clean normal training data: One-Class SVM, autoencoder, PCA reconstruction, or Gaussian boundary methods.

If the data is time-series: forecast residuals, STL + residual rules, change point detection, or sequence models.

If the data is images: autoencoder / embedding-based visual anomaly detection.

If you have labels: frame it as an imbalanced classification problem plus thresholding.

60. Practical workflow for a tabular anomaly project

- 1) Define the anomaly with stakeholders.
- 2) Audit data quality and missingness.
- 3) Build baseline rules and visual checks.

- 4) Train Isolation Forest and LOF.
- 5) Compare score distributions and inspect top anomalies manually.
- 6) Add engineered features and retrain.
- 7) Calibrate threshold using available labels or review capacity.
- 8) Deploy with monitoring dashboard and feedback loop.

61. Practical workflow for time-series monitoring

- 1) Plot the series and identify seasonality, trend, holidays, outages.
- 2) Create baseline model or forecast.
- 3) Score residuals and use dynamic thresholds.
- 4) Add change point detection.
- 5) Evaluate time-to-detect and false alarms.
- 6) Build alert suppression and grouping to avoid duplicate alerts.

62. Python libraries commonly used

scikit-learn: IsolationForest, OneClassSVM, LocalOutlierFactor, EllipticEnvelope, PCA, preprocessing utilities.

PyOD: a large anomaly detection library with many algorithms.

statsmodels: classical time-series modeling and decomposition.

prophet: forecasting-based anomaly workflows.

ruptures: change point detection.

tensorflow / pytorch: autoencoders and deep anomaly models.

63. Minimal scikit-learn examples

64. Isolation Forest example

65. LOF example

66. One-Class SVM example

67. PCA reconstruction example idea

68. Best practices summary

Start simple with interpretable baselines.

Understand the domain before optimizing metrics.

Use visualization and manual review of top anomalies.

Treat thresholding as a product decision, not just a math step.

Expect drift and build retraining/feedback loops.

Measure downstream value: prevented loss, faster response, reduced downtime, better quality.

69. Short comparison of common methods

Z-score / IQR: very simple, univariate, interpretable, weak for complex patterns.

kNN / LOF: intuitive local structure, but scaling can be expensive.

Isolation Forest: strong default for tabular unsupervised data.

One-Class SVM: useful boundary model, but sensitive and less scalable.

PCA / Autoencoder: good when normal data lies on a lower-dimensional manifold or structure can be reconstructed.

Forecast residual methods: excellent for time-series anomalies.

70. Final takeaway

Anomaly detection is not one single algorithm. It is a problem-setting that combines domain knowledge, data preparation, scoring methods, thresholding, evaluation, and operational feedback.

The best systems usually mix: a strong baseline detector, a threshold strategy, explainability, and a human feedback loop.

Code snippets

Isolation Forest

```
from sklearn.ensemble import IsolationForest

model = IsolationForest(
    n_estimators=200,
    contamination=0.02,
    random_state=42
)
model.fit(X_train)

scores = -model.score_samples(X_test) # higher => more anomalous
pred = model.predict(X_test) # -1 anomaly, 1 normal
```

Local Outlier Factor

```
from sklearn.neighbors import LocalOutlierFactor

lof = LocalOutlierFactor(
    n_neighbors=20,
    contamination=0.02
)
pred = lof.fit_predict(X) # -1 anomaly, 1 normal
scores = -lof.negative_outlier_factor_
```

One-Class SVM

```
from sklearn.svm import OneClassSVM

ocsvm = OneClassSVM(kernel='rbf', gamma='scale', nu=0.02)
ocsvm.fit(X_train_normal)

pred = ocsvm.predict(X_test) # -1 anomaly, 1 normal
scores = -ocsvm.score_samples(X_test)
```

PCA reconstruction-error sketch

```
from sklearn.decomposition import PCA
import numpy as np
```

```
pca = PCA(n_components=0.95)
X_train_proj = pca.fit_transform(X_train_normal)
X_train_recon = pca.inverse_transform(X_train_proj)

X_test_proj = pca.transform(X_test)
X_test_recon = pca.inverse_transform(X_test_proj)

recon_error = np.mean((X_test - X_test_recon)**2, axis=1)
```

Interview / exam quick revision sheet

- Anomaly = rare or abnormal behavior relative to expected normal pattern.
- Main setups: supervised, unsupervised, semi-supervised / one-class.
- Main types: point, contextual, collective, local, global.
- Common methods: z-score, IQR, Mahalanobis, KDE, kNN, LOF, DBSCAN, Isolation Forest, One-Class SVM, PCA, Autoencoders.
- For time-series, use residuals, decomposition, change point detection, and sequence models.
- Evaluation: precision, recall, F1, PR-AUC, top-k precision, alert fatigue, business cost.
- Thresholding and explainability are as important as the detector itself.